In [ ]:
```
lambda
```

$pattern-1$:

One argument

In [3]:
```
# create a normal function
def add(num):
    return(num+10)

add
```

Out[3]:
```
<function __main__.add(num)>
```

In [ ]:
```
# what are the arguments : num
# what you are return: num+10
```

In [ ]:
```
<function_name>=lambda <argume_name>:<what you want to return>
# call the function
```

In [4]:
```
add=lambda num:num+10
add(10)
```

Out[4]:
```
20
```

In [ ]:
```
def add(num):
    return(num+10)

add(10)
#########################
add=lambda num:num+10
add(10)
```

In [5]:
```
def cube(num):
    return(num*num*num)

cube=lambda num:num*num*num
cube(10)
```

Out[5]:
```
1000
```

$pattern-2$:

more than one argument

In [6]:
```
def add(num1,num2):
    return(num1+num2)

add=lambda num1,num2:num1+num2
add(20,30)
```

Out[6]:
```
50
```

In [7]:
```
def avg(num1,num2,num3):
    return((num1+num2+num3)/3)

avg(10,20,30)
```

Out[7]: 20.0

In [8]:
```
avg=lambda n1,n2,n3:(n1+n2+n3)/3
avg(10,20,30)
```

Out[8]: 20.0

pattern − 3:

Default argument

In [9]:
```
def avg(num1,num2,num3=30):
    return((num1+num2+num3)/3)

avg(10,20)
```

Out[9]: 20.0

In [ ]:
```
avg=lambda num1,num2,num3=30:(num1+num2+num3)/3
avg(10,20)
```

In [10]:
```
avg=lambda num1,num2=30,num3:(num1+num2+num3)/3
avg(10,20)
```

```
  Cell In[10], line 1
    avg=lambda num1,num2=30,num3:(num1+num2+num3)/3
                                 ^
SyntaxError: non-default argument follows default argument
```

pattern − 4

if-else

In [11]:
```
# write a normal function call
# find the greatest number between two numbers

def greater(n1,n2):
    if n1>n2:
        print(n1)
    else:
        print(n2)
greater(10,20)
```

```
20
```

In [ ]:
```
if n1>n2:
    print(n1)
else:
    print(n2)

[<if_out> <if_con> else <else_out>]

<if_out> <if_con> else <else_out>
```

In [ ]:
```
lambda n1,n2:<<if_out> <if_con> else <else_out>>
```

In [12]:
```
# def greater(n1,n2):
#     if n1>n2:
#         print(n1)
#     else:
#         print(n2)
# greater(10,20)

greater=lambda n1,n2: n1 if n1>n2 else n2
greater(10,100)
```

Out[12]:
```
100
```

In [ ]:
```
# file handling session
# strings
# i will upload text file download
```

In [3]:
```
list1=['hyd','bengaulru','mumbai']
# o/p:['Hyd','Bengaluru','Mumbai']

# apply this by using list comprehenshion
# do this by using for loop

#for i in list1:
    #print(i.capitalize())

[i.capitalize() for i in list1]
```

Out[3]:
```
['Hyd', 'Bengaulru', 'Mumbai']
```

map

In [10]:
```
# when ever you have a list
# if you want to use lambda function

#Q1)  what is your variable : i
#Q2)  what is your output : i.capitalize()
#Q3)  from where you are taking the values : list1
# to continue will map method

# map(lambda <variable>:<output>,<list>)

# map is completed, we need to store the values
# so apply the list

# list(map(lambda <variable>:<output>,<list>))


list1=['hyd','bengaulru','mumbai']

#for i in list1:
    #print(i.capitalize())

list(map(lambda i:i.capitalize(),list1))
```

Out[10]:
```
['Hyd', 'Bengaulru', 'Mumbai']
```

In [8]:
```
for i in list1:
    print(i.capitalize())

[i.capitalize() for i in list1]

list(map(lambda i:i.capitalize(),list1))
```

```
Hyd
Bengaulru
Mumbai
```

Out[8]:
```
['Hyd', 'Bengaulru', 'Mumbai']
```

In [ ]:
```
# l1=[1,2,3,4,5]
# l2=[10,20,30,40,50]
```

In [14]:
```
l1=[1,2,3,4,5]
list(map(lambda i:i*10,l1))
```

Out[14]:
```
[10, 20, 30, 40, 50]
```

In [17]:
```
list1=['hyd','bengaulru','mumbai']
# len(<string>)>3
# out=['benaluru','mumbai']
# you are filtering
# in order to see the output
# when ever conditions are there
# instead of map , use filter

# list(filter(lambda <variable>:<condition>,<list>))

list(map(lambda i: len(i)>3,list1))
```

Out[17]:
```
[False, True, True]
```

In [18]:
```
list1=['hyd','bengaulru','mumbai']
list(filter(lambda i: len(i)>3,list1))
```

Out[18]:
```
['bengaulru', 'mumbai']
```

In [19]:
```
list1=['hy#d','bengaulru','mumbai','che#nnai']
# op=['hy#d','che#nnai']
list(filter(lambda i:'#' in i,list1))
```

Out[19]:
```
['hy#d', 'che#nnai']
```

- lambda
- map
- filter

In [4]:
```
def add(num):
    return(num+10)

add(10)

lambda num:num+10

#lambda <variable>:<output>
```

Out[4]:
```
20
```

In [ ]:

In [ ]:

In [ ]:

In [ ]:

In [ ]: